

Universidad Politécnica de San Luis Potosí

**LAB: FILE PATH TRAVERSAL VALIDATION OF FILE
EXTENSION WITH NULL BYTE BYPASS**

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha del laboratorio	3
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento paso a paso.....	3
Evidencia del procedimiento.....	4
Evidencias obligatorias	7
Explicación del payload.....	9
Mitigación recomendada.....	10

Ficha del laboratorio

Laboratorio: LAB: FILE PATH TRAVERSAL VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

Nivel: Practitioner

Tipo de explotación: File Path Traversal / Directory Traversal con evasión mediante inyección de Byte Nulo (%00)

Objetivo del laboratorio

Explotar una vulnerabilidad de Path Traversal eludiendo la validación de extensión de archivo del servidor mediante la inyección de un byte nulo (%00), con el propósito de leer y extraer el contenido del archivo sensible /etc/passwd del sistema operativo.

Explicación técnica de la vulnerabilidad

Esta vulnerabilidad se origina en una discrepancia de procesamiento entre la capa de validación de la aplicación web y las llamadas de bajo nivel a la API del sistema de archivos del sistema operativo. En este escenario, el backend implementa una regla de control de entrada que verifica que el parámetro proporcionado termine estrictamente con una extensión de archivo permitida (como .jpg o .png). Sin embargo, la aplicación concatena directamente esta entrada en la ruta de búsqueda en disco sin normalizarla previamente.

Al suministrar una secuencia con un terminador de cadena nulo codificado en URL (%00 o byte 0x00), la capa de alto nivel de la aplicación evalúa la cadena completa y confirma que la condición del sufijo se cumple (ej. concluye en .jpg). Posteriormente, cuando la ruta se pasa a las funciones nativas del sistema de archivos (como open() o fopen()), el sistema interpreta el byte nulo como el final absoluto del string (null-terminated string). Por lo tanto, el sistema operativo trunca la ruta en el punto exacto de la inyección e ignora los caracteres posteriores, permitiendo el acceso arbitrario a archivos ajenos al directorio raíz previsto.

En el marco de OWASP Top 10, esta vulnerabilidad se clasifica bajo A01:2021 - Broken Access Control (y secundariamente asociada a fallos de inyección/manejo inseguro de entradas). En la tríada CIA, vulnera de forma crítica la Confidencialidad al permitir la lectura de credenciales, archivos de configuración del sistema y código fuente. Si existiera funcionalidad de escritura, comprometería la Integridad y Disponibilidad mediante sobreescritura de binarios o ejecución remota de código (RCE). En un entorno productivo, esto posibilita la exfiltración de llaves criptográficas, mapeo de cuentas del sistema y escalación horizontal o vertical de privilegios.

Procedimiento paso a paso

Se inició la sesión configurando Burp Suite con la función de interceptación activada en la pestaña Proxy y abriendo el navegador integrado preconfigurado para capturar el tráfico web de la sesión de prueba.

Se navegó a la URL del laboratorio en la tienda en línea para explorar los componentes dinámicos de la interfaz y forzar la carga de los recursos estáticos del catálogo de productos.

Se analizó el registro de tráfico en la pestaña HTTP History, identificando múltiples peticiones dirigidas al endpoint `/image` que utilizaban el parámetro `filename` para solicitar las imágenes correspondientes a cada artículo comercializado.

Se seleccionó una de las solicitudes representativas (`GET /image?filename=49.jpg`) y, mediante el menú contextual, se envió a la herramienta Repeater para realizar pruebas controladas y personalizadas sobre el parámetro vulnerable.

Dentro de Repeater, se ejecutó un envío inicial de la solicitud legítima para confirmar el comportamiento estándar del servidor, el cual respondió con un código HTTP 200 OK y la secuencia binaria correspondiente al formato JPEG de la imagen.

Finalmente, se modificó el valor del parámetro inyectando secuencias de retroceso de directorio junto con el byte nulo antes de la extensión esperada (`../../etc/passwd%0049.jpg`). Al emitir la petición, el servidor eludió la restricción de sufijo y devolvió en el cuerpo de la respuesta el contenido en texto plano del archivo `/etc/passwd`, validando el banner de resolución del laboratorio.

Evidencia del procedimiento

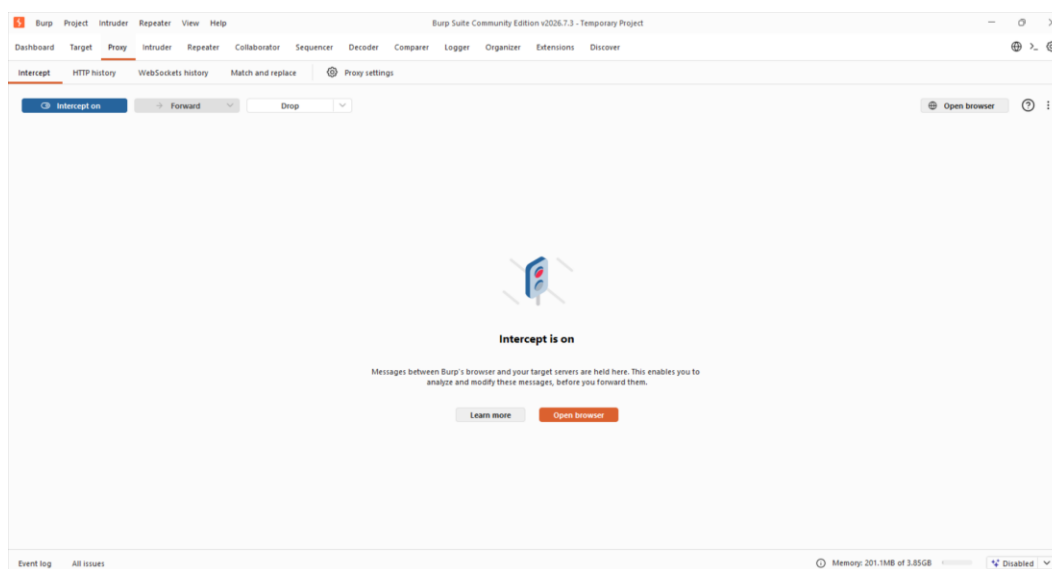


Figura 1. Habilitación del proxy de Burp Suite con la interceptación activa y apertura del navegador integrado.

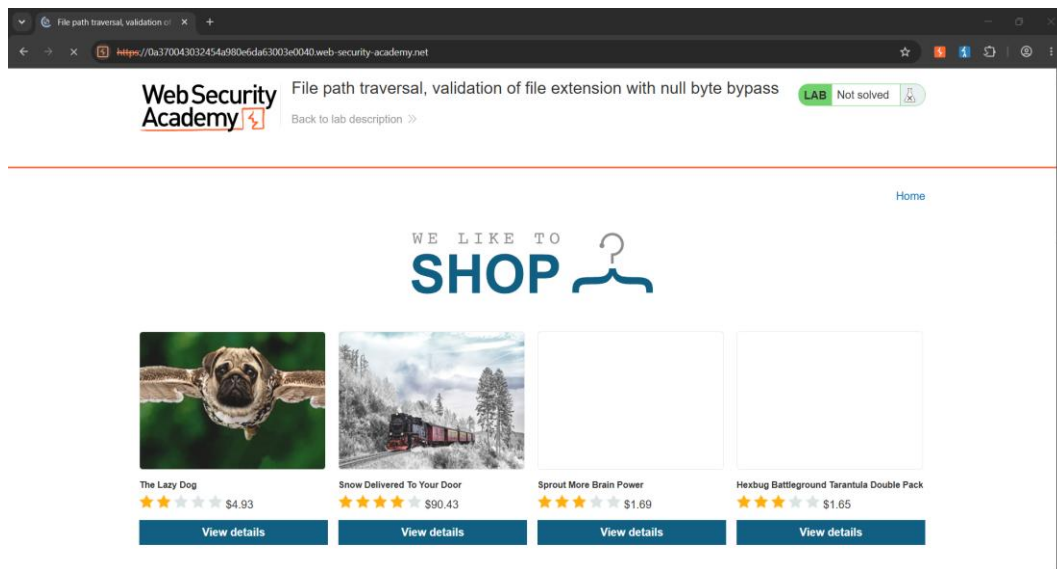


Figura 2. Carga y visualización de la página principal del catálogo de productos dentro de la aplicación objetivo.

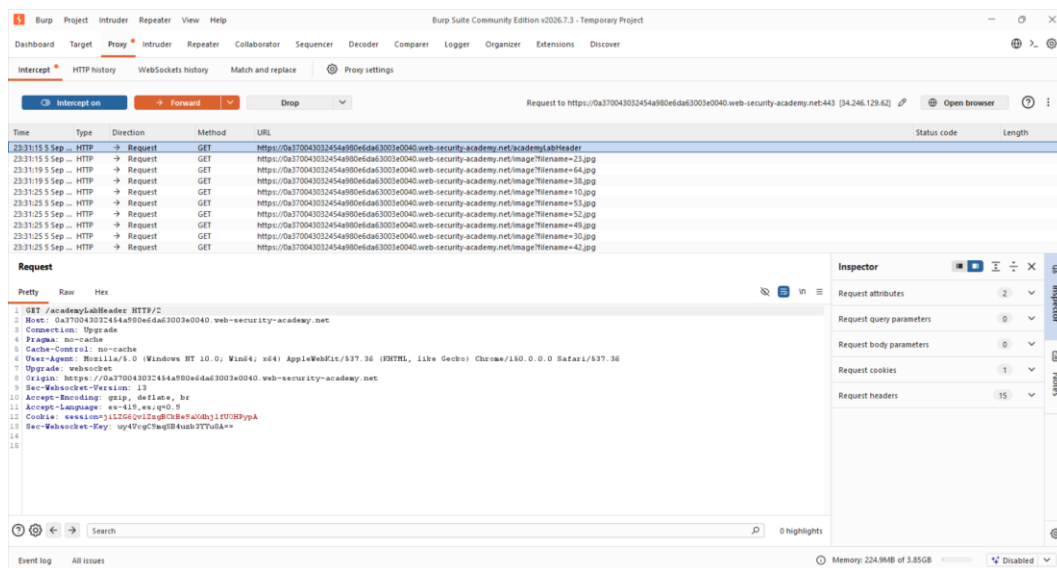


Figura 3. Captura y monitoreo de las peticiones HTTP en el proxy correspondientes a la carga de recursos de imágenes.

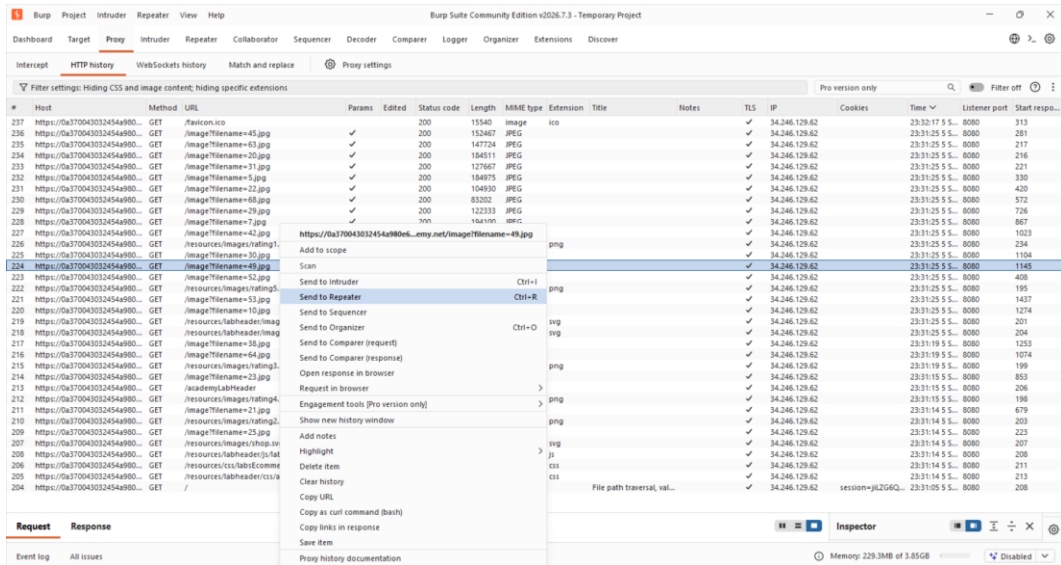


Figura 4. Selección de la solicitud GET a `/image?filename=49.jpg` en el historial HTTP para su reenvío a Repeater.

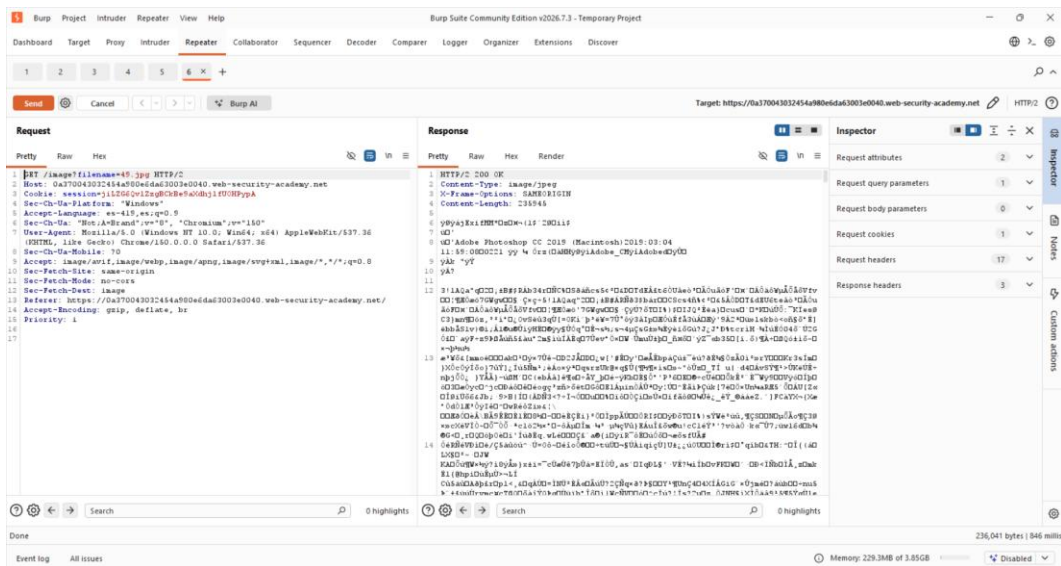


Figura 5. Inspección de la solicitud original en Repeater y validación de la respuesta binaria correspondiente a la imagen.

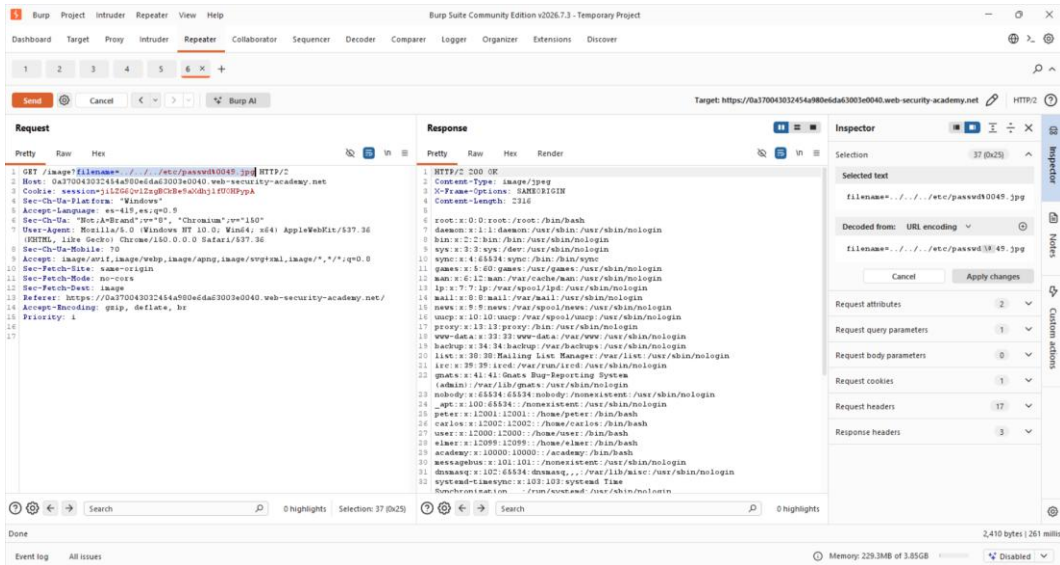


Figura 6. Inyección del payload traversal con byte nulo logrando la exfiltración del archivo /etc/passwd en la respuesta HTTP.

Evidencias obligatorias

Request interceptado

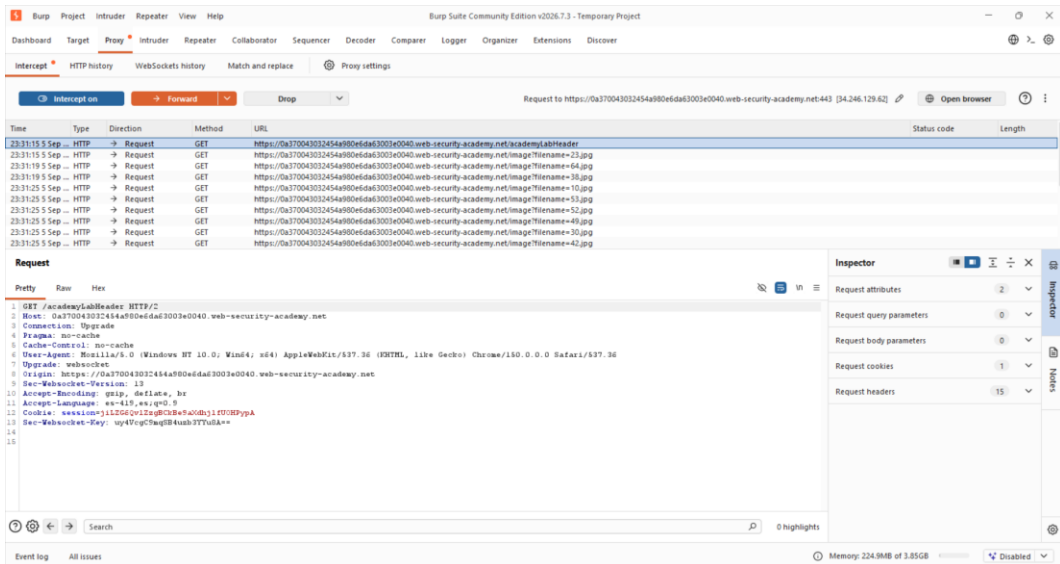


Figura 7. Request interceptado — Petición HTTP interceptada en el Proxy que solicita la carga de un recurso multimedia mediante el endpoint /image.

Payload utilizado



Figura 8. Payload utilizado — Inyección del vector de path traversal con el terminador de byte nulo (%00) en el parámetro filename dentro del Repeater.

Respuesta vulnerable

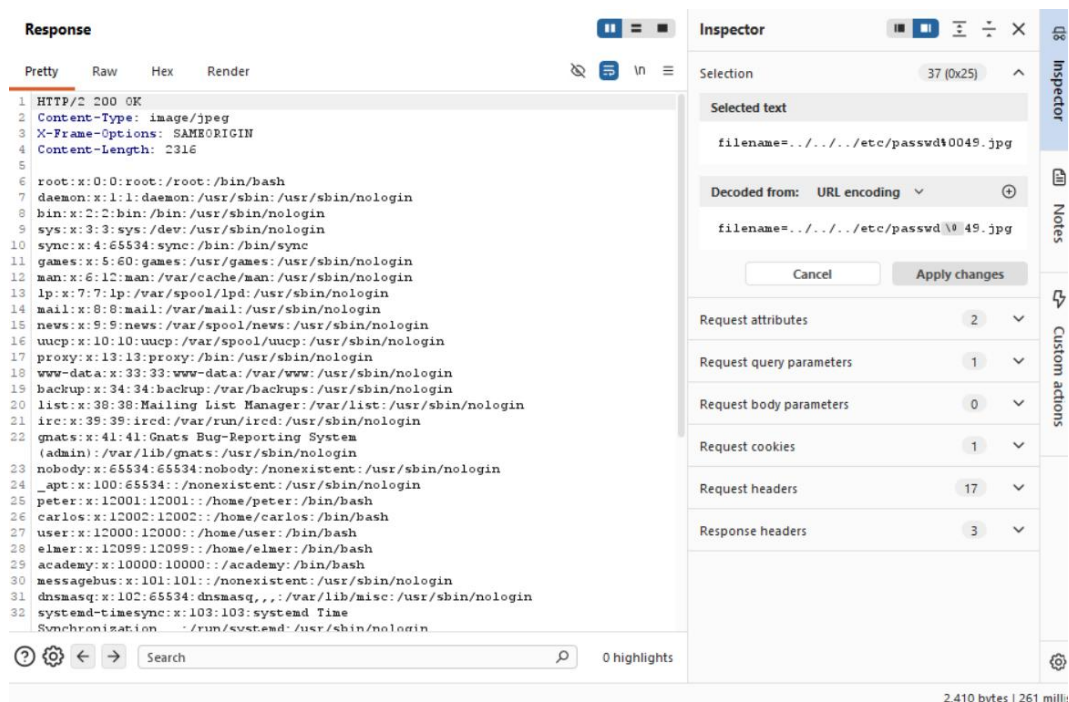


Figura 9. Respuesta vulnerable — Respuesta HTTP 200 OK del servidor mostrando el contenido completo del archivo `/etc/passwd` tras procesar el byte nulo.

Confirmación de “Lab Solved”

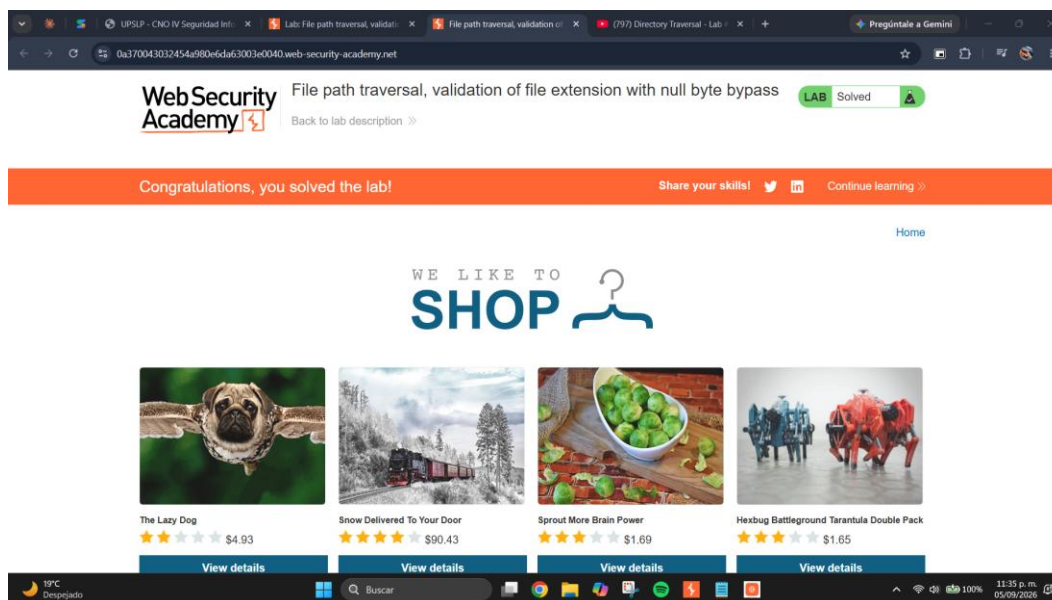


Figura 10. Confirmación de “Lab Solved” — Confirmación de laboratorio resuelto'.

Explicación del payload

El vector de ataque empleado fue: ``../../etc/passwd%0049.jpg`` (o equivalentemente ``../../etc/passwd%00.jpg``).

1. ``../../``: Secuencia de escape de directorios (directory traversal) que retrocede iterativamente en el árbol de directorios del sistema de archivos desde la carpeta base configurada (ej. ``/var/www/images/``) hasta alcanzar el directorio raíz (``/``).
2. ``etc/passwd``: Ruta relativa al archivo objetivo del sistema operativo Unix/Linux que contiene las identidades y configuraciones básicas de las cuentas de usuario locales.
3. ``%00``: Representación URL-encoded del carácter NULL (byte 0x00, '\0'). Funciona como delimitador y terminador de cadena en llamadas nativas del sistema operativo en C. Cuando el sistema operativo procesa la ruta, trunca la lectura exactamente antes de este carácter.
4. ``49.jpg``: Extensión/nombre dummy colocado al final para satisfacer la regla de validación de sufijo aplicada por el backend en capas superiores de código.

Mitigación recomendada

Para mitigar de forma integral esta vulnerabilidad, se deben implementar las siguientes directivas de defensa en profundidad:

1. Desacoplamiento del sistema de archivos: Evitar que parámetros ingresados por el usuario se pasen de manera directa a APIs del sistema operativo. Se recomienda emplear identificadores indirectos (ej. índices numéricos, UUIDs o claves almacenadas en base de datos) para acceder a los archivos.
2. Lista blanca estricta (Whitelisting): Si la entrada del usuario es indispensable, validar que el nombre del archivo se ajuste estrictamente a un conjunto de caracteres alfanuméricos permitidos, rechazando cualquier carácter especial como ``%00``, ``\`` o ````.
3. Canonicalización y verificación de ruta: Resolver la ruta absoluta en el servidor utilizando métodos nativos seguros (como ``getCanonicalPath()`` en Java o ``realpath()`` en PHP/C) y verificar explícitamente que la ruta resultante comience estrictamente con el directorio base permitido antes de abrir o leer el archivo.
4. Actualización del entorno de ejecución: Mantener actualizadas las versiones del lenguaje (ej. versiones modernas de PHP 5.3.4+ o Java que rechazan o mitigan automáticamente la inyección de bytes nulos en llamadas de sistema de archivos).
5. Principio de menor privilegio: Ejecutar el servidor web con una cuenta de usuario restringida que carezca de permisos de lectura sobre archivos sensibles del sistema operativo.